

TP2 - Création de l'Interface de Connexion

Étape 1 : Comprendre les composants de base

Les composants essentiels

- **View** : Conteneur (comme une <div> en HTML)
- **Text** : Afficher du texte
- **TextInput** : Champ de saisie
- **TouchableOpacity** : Bouton tactile
- **StyleSheet** : Gérer les styles

Étape 2 : Créer le fichier de connexion

Créez un nouveau fichier LoginScreen.js

```
import React, { useState } from 'react';

import {
  View,
  Text,
  TextInput,
  TouchableOpacity,
  StyleSheet,
  KeyboardAvoidingView,
  Platform
} from 'react-native';

export default function LoginScreen() {
  // État pour stocker les valeurs des champs
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");

  // Fonction appelée lors du clic sur le bouton
  const handleLogin = () => {
```

```

console.log('Email:', email);
console.log('Password:', password);

// Pour l'instant, on affiche juste dans la console
alert('Tentative de connexion !');
};

return (
  <KeyboardAvoidingView
    behavior={Platform.OS === "ios" ? "padding" : "height"}
    style={styles.container}
  >
    <View style={styles.innerContainer}>
      {/* Titre */}
      <Text style={styles.title}>Connexion</Text>

      {/* Champ Email */}
      <View style={styles.inputContainer}>
        <Text style={styles.label}>Email</Text>
        <TextInput
          style={styles.input}
          placeholder="exemple@email.com"
          value={email}
          onChangeText={setEmail}
          keyboardType="email-address"
          autoCapitalize="none"
        />
      </View>

      {/* Champ Mot de passe */}

```

```

<View style={styles.inputContainer}>
  <Text style={styles.label}>Mot de passe</Text>
  <TextInput
    style={styles.input}
    placeholder="Votre mot de passe"
    value={password}
    onChangeText={setPassword}
    secureTextEntry={true}
  />
</View>

{/* Bouton de connexion */}
<TouchableOpacity
  style={styles.button}
  onPress={handleLogin}
>
  <Text style={styles.buttonText}>Se connecter</Text>
</TouchableOpacity>

{/* Lien vers inscription */}
<TouchableOpacity style={styles.linkContainer}>
  <Text style={styles.linkText}>
    Pas encore de compte ? S'inscrire
  </Text>
</TouchableOpacity>
</View>
</KeyboardAvoidingView>
);
}

```

```
// Styles de l'écran

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#f5f5f5',
  },
  innerContainer: {
    flex: 1,
    justifyContent: 'center',
    paddingHorizontal: 20,
  },
  title: {
    fontSize: 32,
    fontWeight: 'bold',
    textAlign: 'center',
    marginBottom: 40,
    color: '#333',
  },
  inputContainer: {
    marginBottom: 20,
  },
  label: {
    fontSize: 16,
    marginBottom: 8,
    color: '#666',
  },
  input: {
    borderWidth: 1,
```

```
borderColor: '#ddd',
borderRadius: 8,
padding: 15,
fontSize: 16,
backgroundColor: 'white',
},
button: {
  backgroundColor: '#007AFF',
  borderRadius: 8,
  padding: 15,
  alignItems: 'center',
  marginTop: 20,
},
buttonText: {
  color: 'white',
  fontSize: 18,
  fontWeight: 'bold',
},
linkContainer: {
  marginTop: 20,
  alignItems: 'center',
},
linkText: {
  color: '#007AFF',
  fontSize: 16,
},
});
```

Étape 3 : Modifier App.js pour afficher LoginScreen

```
import { StatusBar } from 'expo-status-bar';
```

```
import { SafeAreaView, StyleSheet } from 'react-native';
import LoginScreen from './LoginScreen';
```

```
export default function App() {
  return (
    <SafeAreaView style={styles.container}>
      <LoginScreen />
      <StatusBar style="auto" />
    </SafeAreaView>
  );
}
```

```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#f5f5f5',
  },
});
```

Comprendre le code

useState Hook

```
const [email, setEmail] = useState("");
```

- email : valeur actuelle
- setEmail : fonction pour modifier la valeur
- " : valeur initiale

TextInput Props

- placeholder : texte indicatif
- value : valeur contrôlée
- onChangeText : fonction appelée quand le texte change
- keyboardType : type de clavier (email, numeric, etc.)

- `secureTextEntry` : masque le texte (pour les mots de passe)
- `autoCapitalize` : gestion des majuscules

KeyboardAvoidingView

Empêche le clavier de cacher les champs de saisie :

- iOS : utilise "padding"
- Android : utilise "height"

Étape 4 : Améliorer l'expérience utilisateur

Ajouter la validation des champs

Modifiez la fonction `handleLogin` :

```
const handleLogin = () => {  
  // Validation simple  
  if (!email || !password) {  
    alert('Veuillez remplir tous les champs');  
    return;  
  }  
  
  if (!email.includes('@')) {  
    alert('Veuillez entrer un email valide');  
    return;  
  }  
  
  if (password.length < 6) {  
    alert('Le mot de passe doit contenir au moins 6 caractères');  
    return;  
  }  
  
  // Si tout est ok  
  alert('Connexion réussie !');  
};
```

Ajouter des icônes (optionnel)

Installez une bibliothèque d'icônes :

```
npm install @expo/vector-icons
```

Puis importez et utilisez :

```
import { Ionicons } from '@expo/vector-icons';
```

```
// Dans le JSX, avant le TextInput
```

```
<Ionicons name="mail-outline" size={20} color="#666" />
```

Étape 5 : Gérer l'état de chargement

Ajouter un état de chargement

```
const [isLoading, setIsLoading] = useState(false);
```

```
const handleLogin = async () => {
```

```
  setIsLoading(true);
```

```
  // Simuler un appel API
```

```
  setTimeout(() => {
```

```
    setIsLoading(false);
```

```
    alert('Connexion réussie !');
```

```
  }, 2000);
```

```
};
```

Modifier le bouton

```
<TouchableOpacity
```

```
  style={[styles.button, isLoading && styles.buttonDisabled]}
```

```
  onPress={handleLogin}
```

```
  disabled={isLoading}
```

```
>
```

```
<Text style={styles.buttonText}>
```

```
  {isLoading ? 'Connexion...' : 'Se connecter'}
```



```
</Text>
```

```
</TouchableOpacity>
```

Ajouter le style

```
buttonDisabled: {  
  backgroundColor: '#ccc',  
},
```

Exercices pratiques

1. Ajouter un bouton "Mot de passe oublié ?"

- Créez un nouveau TouchableOpacity
- Ajoutez un style approprié

2. Améliorer la validation

- Vérifiez le format de l'email avec une regex
- Affichez les erreurs sous les champs

3. Ajouter un indicateur de force du mot de passe

- Changez la couleur selon la longueur
- Affichez des conseils

Styles avancés

Ombres (iOS et Android)

```
shadowBox: {  
  // iOS  
  shadowColor: '#000',  
  shadowOffset: { width: 0, height: 2 },  
  shadowOpacity: 0.1,  
  shadowRadius: 4,  
  // Android  
  elevation: 5,  
}
```

Dégradés (avec expo-linear-gradient)

expo install expo-linear-gradient

Dépannage

Le clavier cache les champs

- Vérifiez que KeyboardAvoidingView englobe tout
- Ajustez le behavior selon la plateforme

Les styles ne s'appliquent pas

- React Native n'utilise pas toutes les propriétés CSS
- Consultez la documentation pour les propriétés supportées

Pour aller plus loin

- Animations avec Animated API
- Gestion du focus entre les champs
- Intégration avec un backend